Unrestricted Access to a Critical Method

Unrestricted Access to a Critical Method is a fundamental and highly impactful vulnerability in the smart contract field. The main idea behind it is that smart contracts often contain sensitive methods meant only for specific administrative roles, but fail to properly verify the caller's authorization. This vulnerability occurs when an arbitrary user successfully executes a method that performs a critical operation, even though they are not actually authorized to do so.

The classical example here is a contract with a function designed to modify ownership, mint new tokens, or even invoke `selfdestruct` to destroy the contract entirely. If these functions lack proper access control guards (like an `onlyOwner` modifier), a malicious user can simply call the unprotected function. When this happens, the unauthorized execution can easily lead to a permanent takeover of the contract, the manipulation of pricing mechanisms, or a complete loss of funds.

Specification primitives Our defect specifications rely on standard Solidity datatypes— Contract, Method, Address, Env(ironment), and State. A brief description of the necessary predicates and functions over these types is included in Table 1. The evaluation of these primitives is stratified: structural properties are derived statically via Slither, while behavioral properties are verified at runtime via Forge/Kontrol assertions within the LLM-generated test harness.

Table 1. Primitives for Unrestricted Access to a Critical Method Specification

Symbol	Semantics
Static Analysis (Slither)	
$\mathsf{methods}(c)$	The set of public/external methods defined in contract c.
$\mathsf{IsCritical}(\sigma)$	Holds if the operation executed in state σ is critical (e.g., `selfdestruct`).
Runtime Verification (LLM-generated based on specific template)	
$\mathsf{Exec}(m, \sigma, \sigma')$	Holds if invoking m in state σ successfully transitions to σ'.
$\mathsf{Authorized}(\mathit{Role}, m)$	A mapping that identifies which roles are conceptually intended to access method m. A vulnerability exists if $\mathcal{P}$ grants access to an address that does not map to an authorized *Role*.

Domain Conceptual Model. A contract c is vulnerable to *Unrestricted Access to a Critical Method* if there exists a method m in c and a state σ such that

1. the caller of m (sender) is not authorized to destroy the contract, and

2. the call of m is executed successfully in state σ, and
3. the method m performs a critical operation.

This vulnerability often leads to loss of funds, permanent takeover of the contract, or manipulation of pricing mechanisms.

Formal Specification.

$$\forall m \in \mathsf{methods}(c).\forall \mathit{Role} \in \mathcal{R}.\forall A \in \mathsf{Address}.$$
$$\mathsf{FaultyAccess}(c) \equiv \exists \sigma, \sigma' \in \mathsf{State}.\mathcal{P}(A, m, \sigma) \implies (\mathsf{Exec}(m, \sigma, \sigma') \wedge$$
$$\mathsf{IsCritical}(\sigma')) \wedge (A \notin \mathsf{Authorized}(\mathsf{Role}, m))$$

where
$\mathcal{P} : \mathsf{Address} \times \mathsf{Method} \times \mathsf{State} \to \mathbb{B}$ A dynamic function that determines if an address is authorized to invoke a specific method given the current contract state. This allows for the formalization of flaws in custom modifier logic or uninitialized ownership.
$\mathcal{R}$: The set of all uniquely identified roles (e.g., ADMIN, MINTER, OWNER) defined within the contract's system.

This treats authorization as a dynamic function of the state, capturing flaws in custom logic, uninitialized owners, or flawed role-delegation (e.g., an admin accidentally granting "Owner" rights to a "Minter").

Specific test template design.

Testing Goal Verify if critical methods are properly protected by ensuring that unauthorized addresses cannot reach or modify a sensitive contract state.

Setup The framework identifies all *sensitive* methods—such as those modifying ownership, minting tokens, or invoking `selfdestruct`—using static analysis primitives like `IsCritical(m)`.

Fuzzing The test declares a fuzzed or symbolic address `unauthorizedUser` and applies a constraint using `vm.assume(unauthorizedUser != authorized_address)` to ensure the attacker is not part of the privileged set.

Action The harness uses `vm.prank(unauthorizedUser)` to simulate the transaction from the perspective of the unauthorized entity and attempts to invoke the sensitive method.

Assertion If the sensitive method executes successfully without reverting, or if the contract state (e.g., the `owner` variable) is modified, the access control logic is confirmed as flawed.

Template Derivation and LLM Instantiation The conceptual test design is materialized into a standardized Foundry test template. To bridge the gap between static analysis and generation, the template embeds explicit `[LLM_INSTRUCTION]` comments to guide the LLM through the instantiation phase. The process unfolds as follows:

Target Initialization (Setup): The LLM is instructed to import the target smart contract artifact and initialize the contract under test. The framework identifies sensitive methods—such as those modifying ownership, minting tokens, or invoking `selfdestruct`—as targets for access control validation. It also configures any necessary pre-conditions, such as funding the caller if the targeted method is payable, or setting up required state variables.

Input Parameterization (Fuzzing): To support fuzzing or symbolic execution, the LLM parameterizes the test function with an arbitrary `caller` address and any necessary function arguments. Crucially, the template guides the LLM to tightly constrain this `caller` using `vm.assume`. Beyond excluding test internals and the zero address, the LLM must explicitly exclude ALL privileged roles (e.g., `vm.assume(caller != _contractUnderTest.owner())`) to guarantee the transaction originates from a strictly unauthorized entity.

Exploit Simulation (Action): The LLM synthesizes the action phase by utilizing Foundry cheatcodes. Specifically, it uses `vm.prank(caller)` to switch the execution context, simulating the transaction directly from the perspective of the fuzzed, unauthorized user.

Standardized Validation (Assertion): The test framework leverages the default behavior of the Ethereum Virtual Machine (EVM) for validation. The LLM simply triggers the sensitive method. If the contract is unprotected, the unauthorized call succeeds and the test passes, confirming the vulnerability. If the contract is secure, the call reverts, causing the test to fail. Additionally, the LLM can optionally include assertions to verify specific state changes—such as confirming that the `owner` variable was actually modified by the arbitrary caller—further proving that the access control logic is flawed.

```
1  // SPDX-License-Identifier: MIT
2  pragma solidity 0.8.29;
3
4  import {Test} from "../lib/forge-std/src/Test.sol";
5  // [LLM_INSTRUCTION]: Import the artifact of the contract being tested. The
       solidity files are in "../src/". The name of the file is the same as the
       name of the contract.
6  // [LLM_INSTRUCTION]: If you need to manipulate private state directly,
       import StdStorage:
7  // import {stdStorage, StdStorage} from "../lib/forge-std/src/StdStorage.sol
       ";
8
9  // ------------------------------
10 // [Testing Goal] Verify if critical methods are properly protected by
       ensuring
11 // that unauthorized addresses cannot reach or modify a sensitive contract
       state.
12 // ------------------------------
13
14 // [LLM_INSTRUCTION]: Name the contract 'TestAccessControl[ContractName]'
15 contract TestAccessControlTemplate is Test {
16     // [LLM_INSTRUCTION]: Use StdStorage if needed for complex state setup:
           using stdStorage for StdStorage;
17
18     // ------------------------------
```

```
    // [Setup] Identify and declare the contract under test. The sensitive methods
    // (e.g., modifying ownership, minting tokens, invoking selfdestruct) are the
    // targets for access control validation.
    // ------------------------------

    // [LLM_INSTRUCTION]: Declare the contract under test variable
    // UnprotectedSelfdestruct public _contractUnderTest;

    // ------------------------------  [/Setup]

    function setUp() public {
        // ------------------------------
        // [Setup] Initialize the contract under test.
        // ------------------------------

        // [LLM_INSTRUCTION]: Initialize the contract under test.
        // 1. If constructor parameters are needed, use concrete valid values.
        // 2. If payable, use vm.deal(address(this), amount) before deployment.

        // _contractUnderTest = new UnprotectedSelfdestruct();

        // ------------------------------  [/Setup]
    }

    // [LLM_INSTRUCTION]: Add Fuzz/Symbolic arguments.
    // 1. 'caller': The arbitrary address attempting the access.
    // 2. 'fuzzArg': Any arguments required by the function itself.
    // Example: function test_highlightArbitraryUserCanAccess(address caller, uint256 fuzzArg) public {
    function test_highlightArbitraryUserCanAccess(address caller) public {

        // ------------------------------
        // [Fuzzing] Declare a fuzzed/symbolic address (unauthorizedUser) and
        // constrain it using vm.assume to exclude all privileged addresses,
        // ensuring the caller is not part of the authorized set.
        // ------------------------------

        // [LLM_INSTRUCTION]: Constrain the 'caller'.
        // 1. Caller cannot be the test contract itself.
        vm.assume(caller != address(this));
        // 2. Exclude the Zero Address
        vm.assume(caller != address(0));

        // 3. Exclude Foundry Internals (Dynamic)
        // This catches the 'FoundryCheat' address reliably
        vm.assume(caller != address(vm));
        // Exclude the Console address
        vm.assume(caller != 0x000000000000000000636F6e736F6c652e6c6f67);

        // [LLM_INSTRUCTION]: CRITICAL - Exclude ALL privileged roles.
        // Analyze the contract to find ALL addresses that ARE allowed to call this function.
        // You must exclude them so that vm.assume(unauthorizedUser != authorized_address)
        // guarantees the caller is not part of the privileged set.
        // - If `onlyOwner`: exclude owner.
```

```
72          // - If `msg.sender == vault`: exclude vault.
73          // - If `hasRole(DEFAULT_ADMIN_ROLE, msg.sender)`: exclude admins.
74          // Example:
75          // vm.assume(caller != _contractUnderTest.owner());
76
77          // [LLM_INSTRUCTION]: Constrain other fuzz args if present.
78          // vm.assume(fuzzArg < 100);
79
80          // ------------------------------  [/Fuzzing]
81
82          // ------------------------------
83          // [Setup] Configure any required pre-conditions (funding, state).
84          // ------------------------------
85
86          // [LLM_INSTRUCTION]: FUNDING
87          // Analyze the function being tested. Does it require sending value (
                payable) or checking balances?
88          // If YES: Fund the caller so they can pay for the value transfer.
89          // Example: vm.deal(caller, 100 ether);
90
91          // If NO: You can skip funding to keep the test minimal.
92
93          // [LLM_INSTRUCTION]: STATE VARIABLES
94          // Does the function require specific state to be reachable? (e.g.
                contract not paused).
95          // Use public setters (Strategy A) or vm.store (Strategy B) here.
96
97          // ------------------------------ [/Setup]
98
99          // ------------------------------
100         // [Action] Use vm.prank(unauthorizedUser) to simulate the
                transaction
101         // from the unauthorized entity and attempt to invoke the sensitive
                method.
102         // ------------------------------
103
104         // Switch context to the arbitrary caller
105         vm.prank(caller);
106
107         // ------------------------------  [/Action]
108
109         // ------------------------------
110         // [Assertion] In contrast to standard positive tests, this uses
111         // vm.expectRevert() to assert that the call MUST fail. If the
                sensitive
112         // method executes successfully without reverting, or if the contract
113         // state (e.g., the owner variable) is modified, the access control
114         // logic is confirmed as flawed.
115         // ------------------------------
116
117         // [LLM_INSTRUCTION]: TRIGGER THE SENSITIVE METHOD
118         // Simply call the function.
119         //  If the contract is VULNERABLE (Unprotected): the call SUCCEEDS -
                test PASSES.
120         //  If the contract is SECURE (Protected): the call REVERTS - test
                FAILS.
121
122         // _contractUnderTest.criticalFunction();
123
124         // [LLM_INSTRUCTION]: (Optional) ASSERT STATE CHANGE
```

```
        // Check for side effects to confirm the action really happened.
        // If the sensitive method modifies state (e.g., owner, balances), assert
        // that the change occurred, further proving the access control is flawed.
        // Example: assertEq(_contractUnderTest.owner(), caller);

        // ------------------------------ [/Assertion]
    }
}
```

Listing 1.1. Unrestricted Access to a Critical Method Multi-Shot test template